

```
QSqlQuery query;
QString comm;
comm = "select * from oss";
query.exec(comm);
if(query.isActive()) {
    qDebug() << "Query is Active";
}
else {
    qDebug() << "Query is inactive" << query.lastError();
}
query.next();
if(query.isValid()) {
    qDebug() << "Query is Valid";
}
else {
    qDebug() << "Query is not Valid" << query.lastError();
}
qDebug() << query.value(0).toString();
```

You can provide the SQL query directly to the *QSqlQuery* constructor, or use a string. There are two important points about the code. First, you can check whether the query is active or not. If the query failed to execute, it won't be active. Second, you can check whether your query result points to a valid record by using *isValid()*. The member function *next()* moves the 'current record' pointer to the next record. Before accessing the result, again make sure that it's valid. The result comes in an array of arrays. The columns' values are accessed as *query.value(column_no).toString()*.

Binding

Sometimes you need to construct the query, such as to insert data from GUI widgets, into the database. The simplest method is to use the *sprintf* function:

```
QString str;
str.sprintf("Insert into oss (sr, size) values (%s", "%d)",
label.text().toAscii().data(), size);
```

In most cases, this code will work perfectly. But suppose you have 25 columns to be inserted at a time, the statement would become huge, and it'll be hard to track errors in it. To handle such situations, Qt provides support for binding parameters to place-holders:

```
query.prepare("INSERT INTO oss (sr, size) VALUES (:sr, :size)");
query.bindValue(":sr", 20);
query.bindValue(":size", 199);
```

Here, we have declared two place-holders in the query, *sr* and *size*. Then, *bindValue()* has been used to supply values to the place-holders. Instead of using named place-holders, you can just use '?', in which case, skip the first parameter to *bindValue()*:



Figure 1: SQL data in tabular form

```
query.prepare("INSERT INTO oss (sr, size) VALUES (?, ?)");
query.addBindValue(30);
query.addBindValue(299);
```

During binding, keep in mind the sequence of the fields; if fields are added to the table and the GUI later, it would be harder to maintain than using named place-holders.

Record

Next comes *QSqlRecord*, which is an interesting class that encapsulates a database record (a row, retrieved from a SELECT query). *QSqlRecord* can be used to retrieve, add and remove fields, and do a lot more, which you can read about at <http://doc.qt.nokia.com/4.7-snapshot/qsqlrecord.html#details>.

Tabular display

As we have seen, you can retrieve data from the database and display it in labels or other widgets. But how do you display it in a tabular form? Qt provides a simple solution—create an SQL query model, and apply it to a table view. I created a GUI application with a *QTableView* widget on the dialogue window. From the constructor code, issue the following commands:

```
model = new QSqlQueryModel(this);
model->setQuery("select distro, version from test1");
ui->tableView->setModel(model);
```

We have created a *QSqlQueryModel* object, set an SQL query to it, and connected it to the *tableView*. The data from the query will be in a 2D array, which will be displayed somewhat like what you can see in Figure 1.

That was very simple, right? That's the power of Qt. This article is only an introduction to Qt's database handling; the Qt documentation will be very helpful if you want to learn more. 

By: Manoj Kumar

The author is an open source developer and trainer. You can reach him at manoj@ossilabs.org and to share your knowledge, join his FB group <http://www.facebook.com/groups/nxlt.karnal>.